

NAMA : Ilyas Najwan Pratama
NIM : J0403251081

- Kode program Python

```
1 # =====
2 # 1. STRUKTUR DASAR NODE
3 # =====
4 class Node:
5     # Konstruktor untuk menginisialisasi node baru
6     def __init__(self, data):
7         self.left = None # Pointer untuk menyimpan sub-tree kiri
8         self.right = None # Pointer untuk menyimpan sub-tree kanan
9         self.data = data # Nilai/data yang disimpan pada node saat ini
10
11 # =====
12 # 2. MEKANISME PENAMBAHAN DATA (BINARY SEARCH TREE)
13 # =====
14 def insert(self, data):
15     # Mengecek apakah node saat ini sudah memiliki nilai
16     if self.data:
17         # Jika data baru lebih kecil, arahkan ke sub-tree kiri
18         if data < self.data:
19             if self.left is None:
20                 self.left = Node(data) # Buat node baru jika kiri kosong
21             else:
22                 self.left.insert(data) # Lakukan rekursi ke node kiri
23
24         # Jika data baru lebih besar, arahkan ke sub-tree kanan
25         elif data > self.data:
26             if self.right is None:
27                 self.right = Node(data) # Buat node baru jika kanan kosong
28             else:
29                 self.right.insert(data) # Lakukan rekursi ke node kanan
30         else:
31             self.data = data # Inisialisasi data jika node benar-benar kosong
32
33
```

```
34 # =====
35 # 3. MEKANISME PEMBACAAN TREE (TRAVERSAL)
36 # =====
37
38 # Traversal In-order (Kiri -> Root -> Kanan)
39 # Menghasilkan list data yang terurut dari nilai terkecil ke terbesar (Ascending)
40 def inorderTraversal(root, result=None):
41     if result is None:
42         result = []
43
44     if root:
45         inorderTraversal(root.left, result) # 1. Menelusuri sub-tree kiri secara rekursif
46         result.append(root.data) # 2. Mencatat/menyimpan data node saat ini
47         inorderTraversal(root.right, result) # 3. Menelusuri sub-tree kanan secara rekursif
48
49     return result
50
51 # Traversal Pre-order (Root -> Kiri -> Kanan)
52 # Biasa digunakan untuk menyalin struktur tree secara utuh
53 def preorderTraversal(root, result=None):
54     if result is None:
55         result = []
56
57     if root:
58         result.append(root.data) # 1. Mencatat/menyimpan data node saat ini lebih dulu
59         preorderTraversal(root.left, result) # 2. Menelusuri sub-tree kiri secara rekursif
60         preorderTraversal(root.right, result) # 3. Menelusuri sub-tree kanan secara rekursif
61
62     return result
63
64 # Traversal Post-order (Kiri -> Kanan -> Root)
65 # Biasa digunakan untuk menghapus node dari bawah ke atas
66 def postorderTraversal(root, result=None):
67     if result is None:
68         result = []
69
```

```

64 # Traversal Post-order (Kiri -> Kanan -> Root)
65 # Biasa digunakan untuk menghapus node dari bawah ke atas
66 def postorderTraversal(root, result=None):
67     if result is None:
68         result = []
69
70     if root:
71         postorderTraversal(root.left, result) # 1. Menelusuri sub-tree kiri secara rekursif
72         postorderTraversal(root.right, result) # 2. Menelusuri sub-tree kanan secara rekursif
73         result.append(root.data) # 3. Mencatat/menyimpan data node saat ini di paling akhir
74
75     return result
76
77 # =====
78 # MAIN PROGRAM
79 # =====
80 print("Nama : Ilyas Najwan Pratama")
81 print("NIM : J0403251081")
82 print("=" * 60)
83
84 # Menginisialisasi tree dengan nilai root 81 (dari 2 digit terakhir NIM)
85 tree = Node(81)
86
87 # Memasukkan sisa data sesuai aturan pembentukan pada modul
88 data_tambahan = [61, 101, 51, 71, 91, 111, 66]
89 for d in data_tambahan:
90     tree.insert(d)
91
92 # Memanggil dan mencetak hasil traversal
93 print(f"In-order Traversal : {inorderTraversal(tree)}")
94 print(f"Pre-order Traversal : {preorderTraversal(tree)}")
95 print(f"Post-order Traversal : {postorderTraversal(tree)}")
96

```

- Screenshot hasil eksekusi program, dengan menampilkan NAMA dan NIM pada awal output program

```

76
77 # =====
78 # MAIN PROGRAM
79 # =====
80 print("Nama : Ilyas Najwan Pratama")
81 print("NIM : J0403251081")
82 print("=" * 60)
83
84 # Menginisialisasi tree dengan nilai root 81 (dari 2 digit terakhir NIM)
85 tree = Node(81)
86
87 # Memasukkan sisa data sesuai aturan pembentukan pada modul

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS C:\Users\User\OneDrive\Documents\Tugas IPB\Tugas TPL\Algoritma dan Struktur Data> & C:/Users.
b/Praktikum10/Praktikum10_J0403251081_Ilyas Najwan Pratama.py"
Nama : Ilyas Najwan Pratama
NIM : J0403251081
=====
In-order Traversal : [51, 61, 66, 71, 81, 91, 101, 111]
Pre-order Traversal : [81, 61, 51, 71, 66, 101, 91, 111]
Post-order Traversal : [51, 66, 71, 61, 91, 111, 101, 81]
PS C:\Users\User\OneDrive\Documents\Tugas IPB\Tugas TPL\Algoritma dan Struktur Data>

```

- Hasil traversal:

- In-order

```
In-order Traversal : [51, 61, 66, 71, 81, 91, 101, 111]
```

- Pre-order

```
Pre-order Traversal : [81, 61, 51, 71, 66, 101, 91, 111]
```

- Post-order

```
Post-order Traversal : [51, 66, 71, 61, 91, 111, 101, 81]
```

- Penjelasan singkat:

- Apa perbedaan ketiga traversal tersebut?

In-order Traversal: Menelusuri sub-tree kiri terlebih dahulu, kemudian mengunjungi root, dan terakhir menelusuri sub-tree kanan (Left -> Root -> Right). Metode ini secara otomatis akan menghasilkan luaran data yang terurut secara ascending (dari nilai terkecil hingga terbesar).

Pre-order Traversal: Mengunjungi root terlebih dahulu, kemudian menelusuri sub-tree kiri, dan terakhir menelusuri sub-tree kanan (Root -> Left -> Right).

Post-order Traversal: Menelusuri sub-tree kiri, kemudian menelusuri sub-tree kanan, dan mengunjungi root pada tahap paling akhir (Left -> Right -> Root). Proses ini memastikan seluruh *child node* dieksekusi sebelum *parent node*-nya.

- Menurut anda, Traversal mana yang paling mudah digunakan untuk melihat struktur awal tree?

Traversal yang paling mudah digunakan untuk melihat struktur awal tree adalah **Pre-order Traversal**. Hal ini dikarenakan pada Pre-order, root utama dari tree selalu menjadi elemen pertama yang dikunjungi dan dicetak.

- Apakah hasil traversal akan sama jika data dimasukkan dengan urutan berbeda?

Hasil traversal tidak selalu sama, bergantung pada metode yang digunakan:

Pada In-order Traversal, hasilnya akan selalu sama. Karena sifat dasar In-order pada BST adalah membaca data secara terurut (ascending), outputnya akan tetap konsisten selama himpunan elemen data yang dimasukkan sama.

Pada Pre-order dan Post-order Traversal, hasilnya akan berbeda. Urutan proses penambahan data (insert) sangat menentukan di mana sebuah data dialokasikan, apakah ia akan menjadi root, internal node, atau leaf node.